DM587

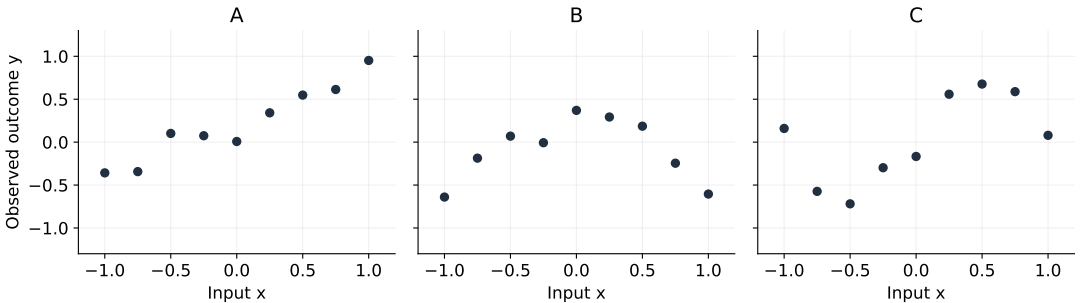
Scientific Programming

Lecture 05: Linear regression

Sai Ganesh Nagarajan

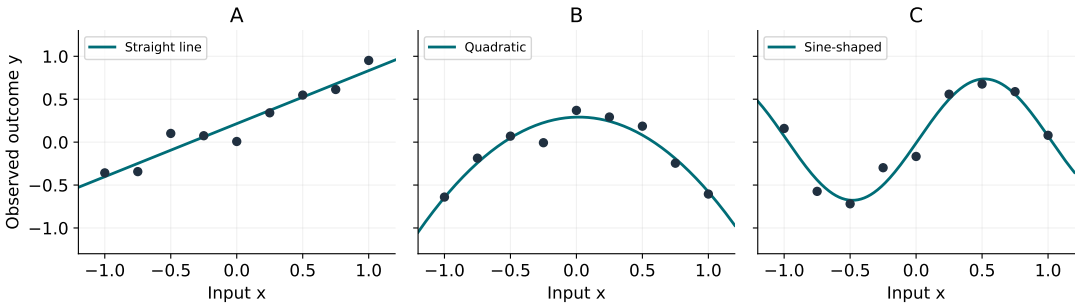
2 November 2026

Which function?



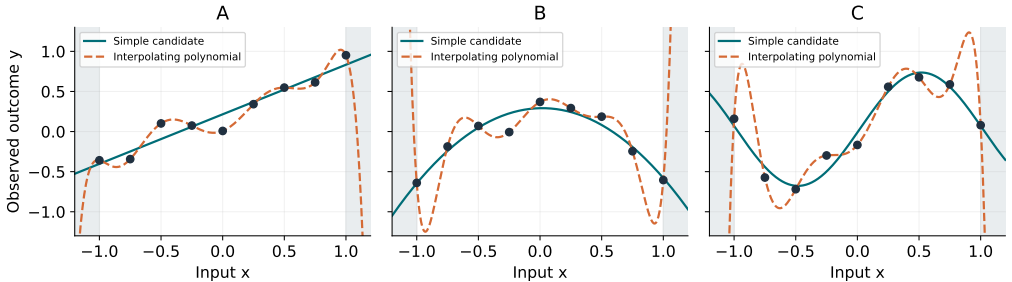
- Suggest a simple function family for each set of points.
- Should the function pass through every point?

Possible choices



- The points are underlying function values plus small simulated noise.
- The curves fit a line, quadratic and sine-shaped family to these noisy observations.

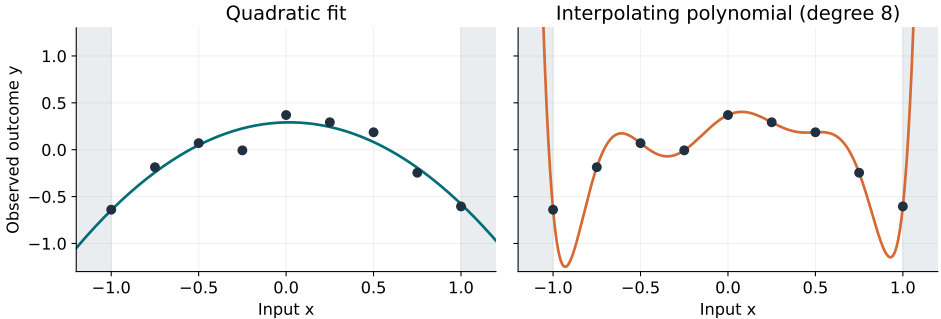
An exact fit



- All three panels also admit a polynomial through **every** point.
- For N distinct scalar inputs, a polynomial of degree at most $N - 1$ can interpolate any N outcomes. Here $N = 9$: degree at most 8.

The grey regions lie beyond the observed inputs. Does zero error at the dots settle which model to choose?

Which would you choose?



Which is the better fit?

The same noisy points. Two different models.

What should decide your choice?

How to choose?

Fit

How close are predictions to the observed outcomes?

New inputs

Does the model predict well on observations it has not fitted?

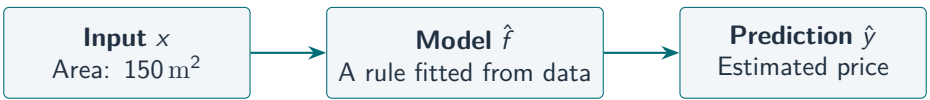
Simplicity

Does the extra flexibility improve useful predictions, or only follow small deviations?

Use knowledge of the problem as well: should the pattern be roughly linear, curved or periodic?

Passing through all the points is not the same as finding a good predictive model.

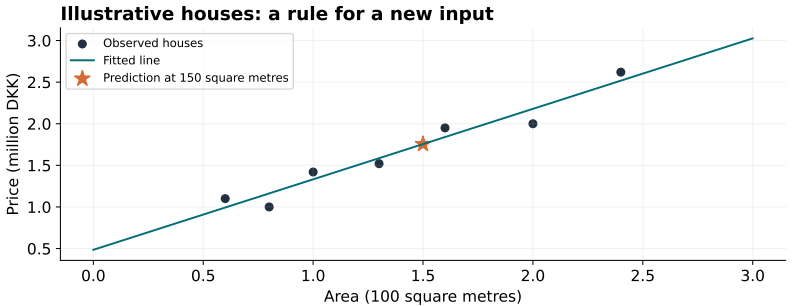
Regression



$$\hat{y} = \hat{f}(x; \theta).$$

- y is the observed outcome; $\hat{y}$ is the model's prediction.
- θ contains the adjustable numbers, called parameters or coefficients.
- Choose a model family, then use past input–outcome pairs to fit θ.

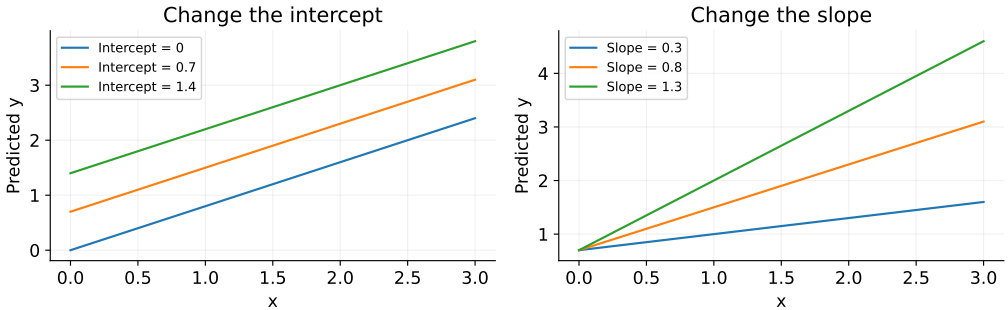
A straight line



$$\hat{y} = \theta_1 + \theta_2 x, \qquad x = \frac{\text{area in square metres}}{100}.$$

y is price in millions of DKK. At $150\,\mathrm{m}^2$, substitute $x = 1.5$ into the fitted rule.

Coefficients



- **Intercept** θ_1: predicted outcome at $x = 0$; shifts the line vertically.
- **Slope** θ_2: change in predicted outcome when x increases by one.

01

Linear systems

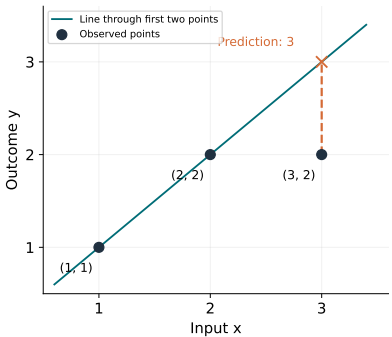
When can every equation be satisfied?

Section reading

Boyd and Vandenberghe, §8.3; §11.3

Exact fitting

Can one line pass through all three points?



For $\hat{y} = \theta_1 + \theta_2 x$, the points require

$$\theta_1 + \theta_2 = 1,$$
$$\theta_1 + 2\theta_2 = 2,$$
$$\theta_1 + 3\theta_2 = 2.$$

The first two force $\theta_1 = 0$, $\theta_2 = 1$: the line is $y = x$.

At $x = 3$, it predicts 3, not 2.

No line satisfies all three equations.

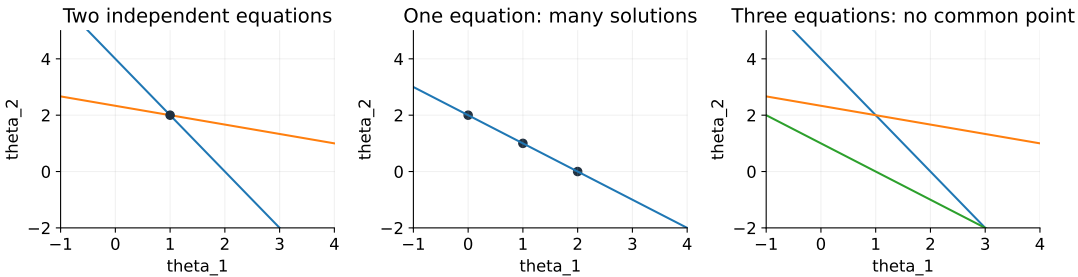
Matrix form

Consider a separate example with two equations in two unknowns:

$$\begin{aligned} 2\theta_1 + \theta_2 &= 4, \\ \theta_1 + 3\theta_2 &= 7 \end{aligned} \quad \Longleftrightarrow \quad \underbrace{\begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}}_{\theta} = \underbrace{\begin{bmatrix} 4 \\ 7 \end{bmatrix}}_{b}.$$

- A contains known coefficients: one row per equation.
- θ contains the unknowns; b contains the required right-hand-side values.
- In general, A has N rows and p columns: N equations for p unknowns.

Equations and unknowns



These plots are in coefficient space: a point is a candidate (θ_1, θ_2); each line is one equation. Exact solutions lie at common intersections.

Rank

For $A\theta = b$, A has N rows and p columns. The cases below assume the stated independence:

Shape	Rank assumption	How many solutions?
Square: $N = p$	$\text{rank}(A) = p$ All columns independent	Exactly one for every b
Tall: $N > p$	$\text{rank}(A) = p$ All columns independent	One if $b \in \text{colsp}(A)$; else none
Wide: $N < p$	$\text{rank}(A) = N$ All rows independent	Infinitely many for every b

Rank counts the independent columns, or equivalently the independent rows.

$$\text{rank}(A) \leq \min(N, p).$$

Singular and nonsingular

These terms distinguish two kinds of **square** matrix:

Nonsingular: independent columns

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \end{bmatrix}.$$

Rank p for a $p \times p$ matrix.

A^{-1} exists.

$A\theta = b$ has exactly one solution for every b.

Singular: dependent columns

$$S = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}.$$

Rank less than p.

S^{-1} does not exist.

$b \in \text{colsp}(S)$: infinitely many solutions.

$b \notin \text{colsp}(S)$: no exact solution.

Rectangular matrices have no ordinary two-sided inverse.

A singular system

Here $\text{colsp}(S) = \text{span}\{(1,2)^T\}$: the right-hand side decides whether a solution exists.

$$S\theta = b, \qquad S = \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}.$$

Consistent: $b = (2,4)$

$$\theta_1 + \theta_2 = 2, \qquad 2\theta_1 + 2\theta_2 = 4.$$

$b = (2,4)^T \in \text{colsp}(S)$.

Infinitely many exact solutions, such as $(0,2)$ and $(1,1)$.

Inconsistent: $b = (2,5)$

$$\theta_1 + \theta_2 = 2, \qquad 2\theta_1 + 2\theta_2 = 5.$$

$b = (2,5)^T \notin \text{colsp}(S)$.

No exact solution exists.

Computational linear algebra

Linear algebra

Does $A\theta = b$ have a solution?

Is the solution unique?

What does rank tell us?

Computation

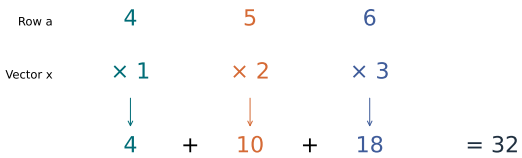
Which operations find the solution?

How much work and memory do they need?

How do rounding and data changes affect the result?

On a computer, an algebraically correct formula is only the starting point. We choose an algorithm that uses the matrix structure.

Dot product



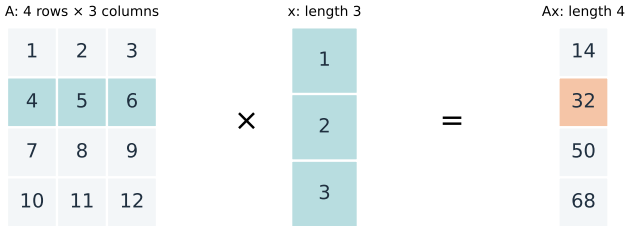
For two length-n vectors,

$$a^T x = \sum_{j=1}^{n} a_j x_j : \qquad n \text{ multiplications} + (n-1) \text{ additions.}$$

- Total: $2n - 1$ arithmetic operations. We describe its growth as $O(n)$.
- $O(n)$ means linear growth: roughly twice the work for twice the length. We ignore constant factors and lower-order terms.

Count arithmetic operations, not lines of Python or elapsed seconds.

Matrix–vector product

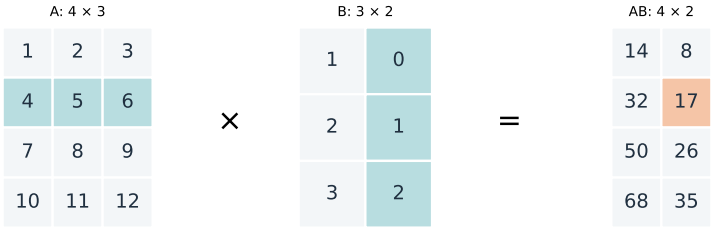


The highlighted output is one row dotted with the vector:

$$(Ax)_2 = 4 \times 1 + 5 \times 2 + 6 \times 3 = 32.$$

- An $m \times n$ matrix has m rows: repeat a length-n dot product m times.
- Work: $m(2n-1) = O(mn)$. For an $n \times n$ matrix: $O(n^2)$.

Matrix–matrix product



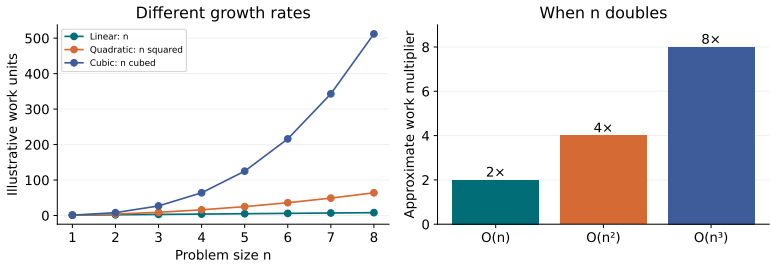
Each output pairs a row of A with a column of B:

$$(AB)_{2,2} = 4 \times 0 + 5 \times 1 + 6 \times 2 = 17.$$

- A is $m \times n$, B is $n \times p$: there are mp output entries, each a length-n dot product.
- Work: $mp(2n-1) = O(mnp)$. Two $n \times n$ matrices: $O(n^3)$.

This counts the standard dense row-by-column algorithm.

Growth



- Dot product: $O(n)$; square matrix–vector: $O(n^2)$; standard square matrix–matrix: $O(n^3)$.
- The curves illustrate growth, not measured timings. Constants, hardware and implementation still matter.

The inverse formula

For a square, nonsingular matrix, linear algebra gives

$$A\theta = b \qquad \Longrightarrow \qquad \theta = A^{-1}b.$$

For example,

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}, \qquad b = \begin{bmatrix} 4 \\ 7 \end{bmatrix}, \qquad A^{-1} = \frac{1}{5}\begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix},$$

$$\theta = \frac{1}{5}\begin{bmatrix} 3 & -1 \\ -1 & 2 \end{bmatrix}\begin{bmatrix} 4 \\ 7 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

- This is a correct formula, but first we must compute the inverse.
- Standard direct solvers instead use elimination, organised as a factorisation, to find θ without forming A^{-1}.

Elimination

Create zeros below the diagonal without changing the solutions:

$$\left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 2 & 3 & 1 & 11 \\ 1 & 2 & 3 & 14 \end{array}\right] \xrightarrow{1} \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 0 & 1 & 2 & 8 \end{array}\right] \xrightarrow{2} \left[\begin{array}{ccc|c} 1 & 1 & 1 & 6 \\ 0 & 1 & -1 & -1 \\ 0 & 0 & 3 & 9 \end{array}\right].$$

1. Cancel the first unknown from rows 2 and 3:

$$R_2 \leftarrow R_2 - 2R_1, \qquad R_3 \leftarrow R_3 - R_1.$$

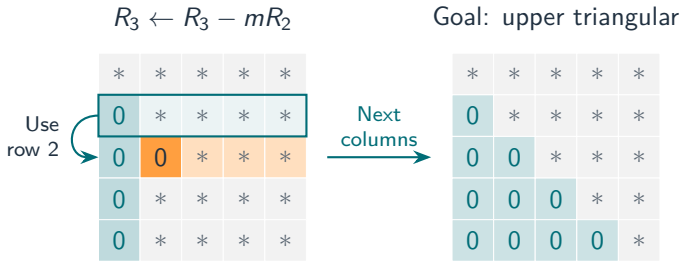
2. Cancel the second unknown from row 3:

$$R_3 \leftarrow R_3 - R_2.$$

R_i means row i. The result is upper triangular; solve it from the bottom upwards. Numerical solvers swap rows when needed (pivoting). For a dense $p \times p$ system, elimination costs $O(p^3)$.

Why? Count the entry updates, not just the zeros created.

Elimination work



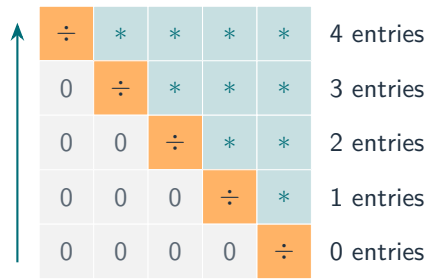
Blue zeros: already made, preserved. Dark orange: new zero. Pale orange: updated entries.

- Earlier zeros stay zero: both rows have 0 in column 1. Work left to right.
- Column 1: $(p-1) \times (p-1)$ updates; then $(p-2) \times (p-2)$, and so on. Each entry uses one multiplication and one subtraction.

$$2((p-1)^2 + (p-2)^2 + \cdots + 1^2) = O(p^3).$$

Triangular solve

Solve from the bottom upwards, using the values already found.



The rows use $0, 1, 2, \ldots, p-1$ **known values**.

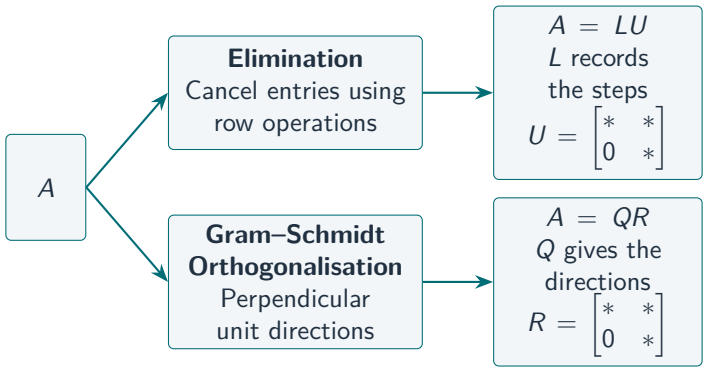
Blue entries: one multiplication and one subtraction each.

Orange diagonal: one division per row.

Visit the triangular region **once**.

$$2\underbrace{(0+1+\cdots+(p-1))}_{p(p-1)/2 \text{ entries}}+\underbrace{p}_{\text{divisions}}=p^2=O(p^2).$$

Two routes



Both routes produce triangular systems that are easy to solve.

The elimination we just performed leads to LU; QR is a separate route.

Factorisation

For dense matrices: first compute the factors, then use them to solve.

LU: general $N \times p$ matrix

$$PA = LU, \qquad k = \min(N, p)$$

L: $N \times k$, lower-triangular pattern.
U: $k \times p$, upper-triangular pattern.

P records row swaps; L stores elimination multipliers. The factors may be rectangular.

Factorisation: $O(Npk)$.

Square nonsingular case:
$O(p^3)$ to factor, then $O(p^2)$ to solve $Lc = Pb$, $U\theta = c$.

QR: general $N \times p$ matrix

$$A = QR, \qquad k = \min(N, p)$$

Q: $N \times k$, orthonormal columns.
R: $k \times p$, upper-triangular pattern.

The factorisation exists for tall, square and wide matrices.

Factorisation: $O(Npk)$.

Tall, independent-column case:
$R\hat{\theta} = Q^T b$ solves least squares. After factoring: $O(Np + p^2)$.

Solving a square system

Use `np.linalg.solve` when A is **square and nonsingular**: find the unique θ satisfying $A\theta = b$.

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 3 & 1 \\ 1 & 2 & 3 \end{bmatrix} \theta = \begin{bmatrix} 6 \\ 11 \\ 14 \end{bmatrix} \quad \Longrightarrow \quad \theta = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

```
import numpy as np
M = np.array([[1., 1., 1.], [2., 3., 1.], [1., 2., 3.]])
b = np.array([6., 11., 14.])
theta = np.linalg.solve(M, b)
print(theta)                          # [1. 2. 3.]
print(np.allclose(np.dot(M, theta), b))  # True
```

A rectangular or singular matrix raises `LinAlgError`; `solve` does not switch to an approximate fit.

Choosing a solver

Why solve directly instead of computing $A^{-1}b$?

- **Storage and work:** forming the full $p \times p$ inverse adds work and storage when we only need one solution vector.
- **Numerics:** computing an inverse and then multiplying can introduce additional rounding error.
- **Existence:** an ordinary inverse exists only for a square, nonsingular matrix.

Shape and rank of A	Python choice
Square: $N = p$, rank p	`np.linalg.solve(A, b)`
Rectangular ($N \neq p$), or square and singular	`np.linalg.lstsq(A, b, rcond=None)[0]`

`lstsq` finds an exact solution when possible; otherwise it minimises the squared residual. If minimisers are nonunique, it selects the smallest-norm coefficients.

Rectangular systems

For $A\theta = b$, first ask whether $b \in \text{colsp}(A)$.

Exact solution exists

$b \in \text{colsp}(A)$: use **elimination** to reduce the equations and solve for the pivot variables.

$$\theta_1 + \theta_2 = 2 \quad \Longrightarrow \quad \begin{cases} \theta_1 = 2 - t, \\ \theta_2 = t. \end{cases}$$

Too few independent constraints leave free variables: here every $t \in \mathbb{R}$ gives a solution. Independent columns give a unique solution.

No exact solution

$b \notin \text{colsp}(A)$: elimination reveals a contradiction. For example,

$$\theta_1 = 1, \quad \theta_2 = 1, \quad \theta_1 + \theta_2 = 3.$$

Instead choose the **least-squares** fit:

$$\min_{\theta} \|A\theta - b\|_2^2.$$

It finds the closest achievable $A\theta$ to b.

In Python, `lstsq` handles both cases: exact when possible, closest fit otherwise. For nonunique solutions, it returns the smallest-norm coefficient vector.

02

Regression and least squares

From exact equations to fitting observed outcomes.

Section reading

Boyd and Vandenberghe, §12.1–12.3; §13.1

Features

A house has more than one useful input:

$$x = \begin{bmatrix} \text{area} \\ \text{bedrooms} \end{bmatrix} \in \mathbb{R}^2, \qquad \hat{y} = \theta_1 + \theta_2 x_1 + \theta_3 x_2.$$

- The feature vector x collects the measured inputs for one house.
- Here there are $n = 2$ input features and $p = 3$ coefficients, including the intercept.
- The model is affine in x: a plane rather than a line. It is linear in θ.

Coefficients describe the fitted relationship with the other inputs held fixed.

Basis functions

Choose p known functions of the input; fit only their coefficients:

$$\hat{f}(x) = \theta_1 f_1(x) + \cdots + \theta_p f_p(x) = f(x)^T \theta.$$

Straight line

$f(x) = (1, x)$

$\theta = (\theta_1, \theta_2)$

Area and bedrooms

$f(x) = (1, x_1, x_2)$

$\theta = (\theta_1, \theta_2, \theta_3)$

The entries of $f(x)$ are basis functions, also called feature mappings. We will later choose powers or sine/cosine functions.

Linear regression means linear in the fitted coefficients; the plotted curve need not be a straight line.

The design matrix

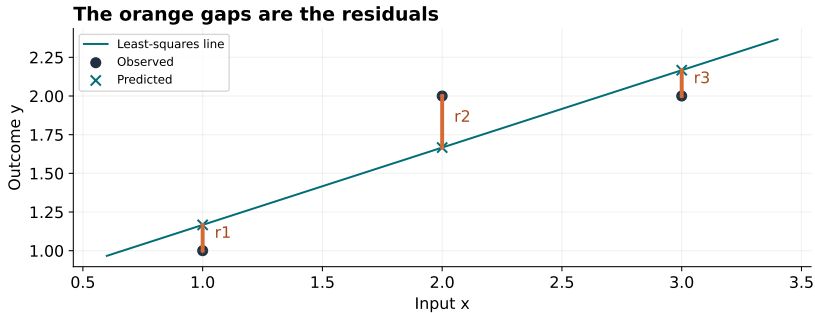
Evaluate every basis function at every observed input:

$$\underbrace{\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}}_{A \in \mathbb{R}^{3\times 2}} \underbrace{\begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix}}_{\theta \in \mathbb{R}^2} = \underbrace{\begin{bmatrix} \hat{y}^{(1)} \\ \hat{y}^{(2)} \\ \hat{y}^{(3)} \end{bmatrix}}_{\hat{y}^d \in \mathbb{R}^3}, \qquad y^d = \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix}.$$

- One **row** per observation; one **column** per basis function.
- In general, $A_{ij} = f_j(x^{(i)})$ and A has shape $N \times p$.
- $\hat{y}^d = A\theta$: one multiplication gives all the predictions.

The superscript d marks a vector for the whole dataset; y without it denotes one scalar outcome.

Residuals



$$r^{(i)} = y^{(i)} - \hat{y}^{(i)}, \qquad r^d = y^d - A\theta.$$

Residuals are vertical gaps: we measure errors in the outcome, holding each input fixed. Positive means the prediction was too low.

Squared error

Choose coefficients to minimise the total squared residual:

$$\hat{\theta} \in \operatorname*{arg\,min}_{\theta \in \mathbb{R}^p} \underbrace{\sum_{i=1}^{N} \left(y^{(i)} - \hat{f}(x^{(i)})\right)^2}_{\text{sum of squared errors, SSE}} = \operatorname*{arg\,min}_{\theta} \|y^d - A\theta\|_2^2.$$

- Squaring prevents positive and negative residuals from cancelling.
- It gives large misses more weight than small ones.
- Every minimiser gives the same fitted vector; coefficients are unique when A has full column rank.

argmin means the coefficient vectors giving the minimum; $\hat{\theta}$ denotes one fitted choice.

Least squares in Python

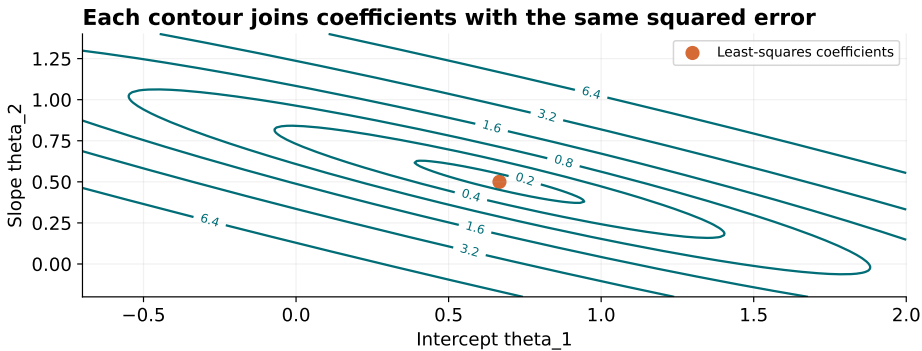
In regression, an exact fit may not exist. Use `np.linalg.lstsq` to minimise the squared error:

$$\hat{\theta} \in \arg\min_{\theta} \|y^d - A\theta\|_2^2.$$

```
# A: design matrix; y: observed outcome vector.
theta, sums, rank, singular = np.linalg.lstsq(A, y, rcond=None)
predicted = np.dot(A, theta)
residual = y - predicted
```

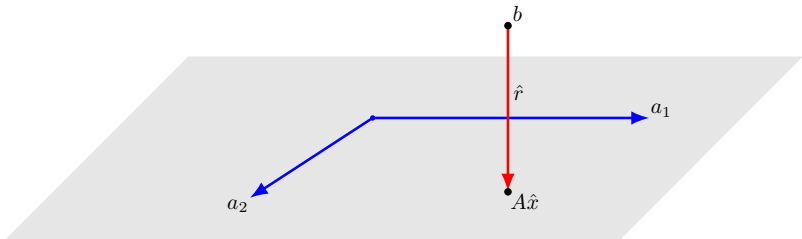
- Works with tall, square or wide matrices, including dependent columns.
- If an exact fit exists, it finds one, up to rounding. Otherwise it finds a best approximate fit.
- If several coefficient vectors minimise the error, it returns the one with smallest norm.

Choosing coefficients



Moving in coefficient space changes the line in the data plot. The minimum chooses the pair with the smallest total squared vertical error.

Projection



- The shaded plane is colsp(A): all combinations of a_1, a_2.
- The closest point to b is $A\hat{x}$; the residual is perpendicular to the plane.

In our notation: $b = y^d$, $\hat{x} = \hat{\theta}$, $A\hat{x} = \hat{y}^d$. The book's residual $\hat{r} = -r^d$.

Figure: Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* (2018), Fig. 12.2, p. 231.

Normal equations

At the closest fitted point, the residual is perpendicular to every column:

$$A^T(y^d - A\hat{\theta}) = 0 \quad \Longleftrightarrow \quad A^T A\hat{\theta} = A^T y^d.$$

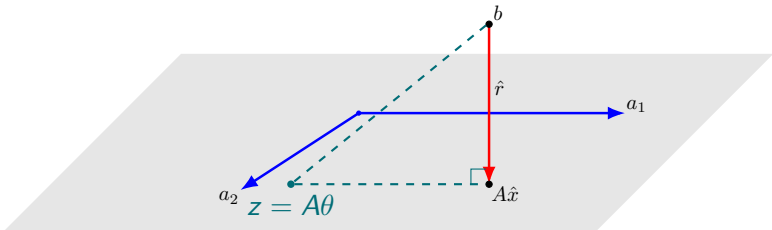
With independent columns, $A^T A$ is invertible, giving the formula $\hat{\theta} = (A^T A)^{-1} A^T y^d$. But we can compute the fit without forming it.

Substitute $A = QR$, with $Q^T Q = I$:

$$R^T \underbrace{Q^T Q}_{I} R\hat{\theta} = R^T Q^T y^d \quad \Longrightarrow \quad \boxed{R\hat{\theta} = Q^T y^d}.$$

- R^T is invertible, so it cancels from both sides.
- Compute $Q^T y^d$, then use back substitution: no $A^T A$ or inverse is needed.
- With dependent columns, the inverse formula is unavailable; `lstsq` still finds a best fit.

Why it is a minimum



For any other fitted vector z in the plane, r^d is perpendicular to $z - \hat{y}^d$. Pythagoras gives

$$\underbrace{\|y^d - z\|_2^2}_{\text{other squared error}} = \underbrace{\|r^d\|_2^2}_{\text{error at the projection}} + \underbrace{\|z - \hat{y}^d\|_2^2}_{\geq 0} \geq \|r^d\|_2^2.$$

Adapted from Boyd & Vandenberghe, *Introduction to Applied Linear Algebra* (2018), Fig. 12.2, p. 231. In our notation, $b = y^d$ and $A\hat{x} = \hat{y}^d$.

Features set the size

$$\underbrace{A^T}_{p\times N}\underbrace{A}_{N\times p} = \underbrace{A^T A}_{p\times p}.$$

Illustrative feature set	p	Dense $A^T A$
10 housing inputs + intercept	11	968 bytes
64×64 RGB pixels + intercept	12,289	1.21 GB
100,000 word features + intercept	100,001	80.0 GB

N counts observations; p counts coefficients. More features increase both dimensions of $A^T A$, giving p^2 entries.

Float64: 8 bytes per entry; decimal GB. Storage shown is for $A^T A$ alone, excluding A, factors, an inverse and workspace.

The Gram matrix $A^T A$

Dense computation	Work or storage
Form $A^T A$	$O(Np^2)$ arithmetic work
Store $A^T A$	$O(p^2)$ numbers
Factorise nonsingular $A^T A$	$O(p^3)$ arithmetic work

- Computing an explicit inverse does additional work for our one outcome vector; avoiding it does not remove all the costs above.
- Forming $A^T A$ can make numerical sensitivity worse, especially when features are nearly dependent.
- A sparse design can produce a much denser $A^T A$, losing the storage advantage of its many zeros.

Practical solvers can work with the original A instead of explicitly building and inverting $A^T A$.

QR in Python

For a tall matrix A with independent columns and observed outcomes y:

$$A = QR \qquad \Longrightarrow \qquad R\hat{\theta} = Q^T y.$$

```
import numpy as np
from scipy.linalg import solve_triangular

Q, R = np.linalg.qr(A, mode="reduced")
c = np.dot(Q.T, y)
theta = solve_triangular(R, c)
```

- Reduced QR keeps only the required p columns of Q. The last line uses back substitution.
- Work: $O(Np^2)$ to factor, then $O(Np + p^2)$ to project and solve. No $A^T A$ or inverse is formed.

When the matrix is singular

If columns of A are dependent, $A^T A$ has no inverse. We can still compute a least-squares fit using the original A:

```
theta, sums, rank, singular = np.linalg.lstsq(A, y, rcond=None)
```

- NumPy's dense `lstsq` uses an **SVD-based solver**: singular value decomposition handles the dependent directions.
- It returns a best fit and, when coefficients are nonunique, chooses the solution with the smallest coefficient norm.
- SVD details come later, alongside eigenvalue decomposition.
- QR can be cheaper than SVD for full-rank problems; both have the same leading order for tall matrices. Actual speed depends on the problem and implementation.

Try-it companion: QR and SVD-based least-squares examples.

A fitted line

Return to the observations $(1,1),(2,2),(3,2)$:

```
x = np.array([1., 2., 3.])
y = np.array([1., 2., 2.])
A = np.column_stack([np.ones(len(x)), x])
theta, sums, rank, singular = np.linalg.lstsq(A, y, rcond=None)
predicted = np.dot(A, theta)
residual = y - predicted
```

$$\hat{\theta} = \begin{bmatrix} 2/3 \\ 1/2 \end{bmatrix}, \qquad \hat{y}^d = \begin{bmatrix} 7/6 \\ 5/3 \\ 13/6 \end{bmatrix}, \qquad r^d = \begin{bmatrix} -1/6 \\ 1/3 \\ -1/6 \end{bmatrix}.$$

Error size

For residuals $(-1/6,\ 1/3,\ -1/6)$ and $N = 3$ observations:

$$\underbrace{\mathrm{SSE}}_{\text{sum of squared errors}} = \sum_i (r^{(i)})^2 = \frac{1}{36} + \frac{1}{9} + \frac{1}{36} = \frac{1}{6}.$$

$$\underbrace{\mathrm{MSE}}_{\text{mean squared error}} = \frac{\mathrm{SSE}}{N} = \frac{1}{18}.$$

$$\underbrace{\mathrm{RMSE}}_{\text{root mean squared error}} = \sqrt{\mathrm{MSE}} = \sqrt{\frac{1}{18}} \approx 0.236.$$

- RMSE has the same units as the observed outcome.
- For a fixed dataset, all three measures choose the same best-fit coefficients.

Similar features

The columns contain almost the same information:

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1.001 \\ 1 & 0.999 \end{bmatrix}.$$

Original outcomes

$$b = \begin{bmatrix} 2 \\ 2.001 \\ 1.999 \end{bmatrix} \implies \hat{\theta} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Tiny change to outcomes

$$b' = \begin{bmatrix} 2 \\ 2.002 \\ 1.998 \end{bmatrix} \implies \hat{\theta}' = \begin{bmatrix} 0 \\ 2 \end{bmatrix}.$$

Changing the outcomes by at most 0.001 changes each coefficient by 1. Nearly identical features can make the coefficients sensitive to small data changes.

Nonunique coefficients

When A is wide or has dependent columns, several coefficient vectors may give the same best fit.

$$\theta_1 + \theta_2 = 2: \qquad (0,2),\ (1,1),\ (2,0) \text{ all fit exactly.}$$

```
A = np.array([[1., 1.]])
y = np.array([2.])
theta = np.linalg.lstsq(A, y, rcond=None)[0]
print(theta)                    # [1. 1.]
```

- `lstsq` chooses the smallest Euclidean coefficient norm among all least-squares solutions.
- This extra rule chooses a vector; it does not create information missing from the observations.

03

Other regression models

Change the features; keep the fitting machinery.

Section reading

Boyd and Vandenberghe, §13.1–13.3

Polynomial regression

A quadratic model is still linear in its three coefficients:

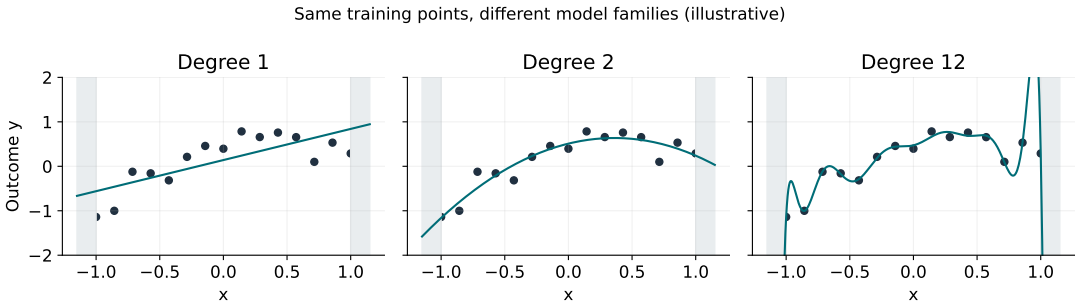
$$\hat{f}(x) = \theta_1 + \theta_2 x + \theta_3 x^2.$$

$$A = \begin{bmatrix} 1 & x^{(1)} & (x^{(1)})^2 \\ 1 & x^{(2)} & (x^{(2)})^2 \\ \vdots & \vdots & \vdots \\ 1 & x^{(N)} & (x^{(N)})^2 \end{bmatrix}, \qquad \hat{y}^d = A\theta.$$

- Degree d uses $p = d + 1$ basis functions: $1, x, \ldots, x^d$.
- The curve bends in input space; fitting the coefficients is the same least-squares problem.

$x^{(i)}$ labels a sample; $(x^{(i)})^2$ squares that sample's input value.

Flexibility



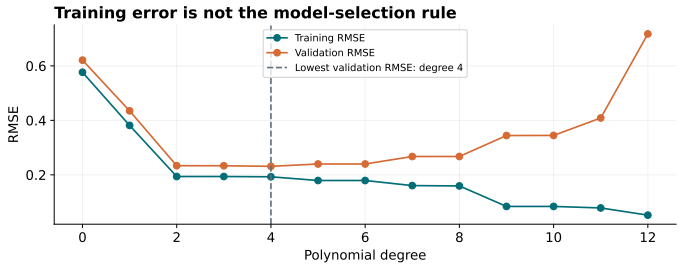
The same illustrative training points are used in every panel. A more flexible family can follow individual errors and behave poorly between or beyond observed inputs.

Polynomial features

```
# x and y contain the training inputs and outcomes.
degree = 2
A = np.column_stack([x ** j for j in range(degree + 1)])
theta = np.linalg.lstsq(A, y, rcond=None)[0]
# Apply the same feature functions to new inputs.
A_new = np.column_stack([x_new ** j
                         for j in range(degree + 1)])
y_new = np.dot(A_new, theta)
```

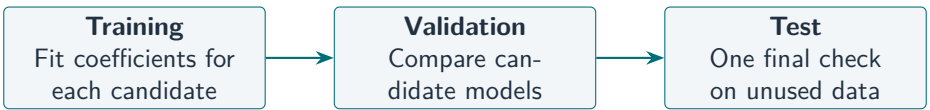
- `x_new` is an array of inputs where we want predictions.
- Feature definitions and any scaling must match training.
- High powers can be badly conditioned; rescale the input or use a better basis.

Model selection



- Degree 4 has the lowest validation RMSE: 0.231. Degrees 2 (0.234) and 3 (0.233) are nearly as good.
- Degree 2 is a reasonable simpler choice: a tiny validation improvement need not justify extra terms.

Occam's razor



- Prefer a simpler model when it predicts comparably well on validation data.
- Simpler models are easier to interpret and often require fewer observations.
- Simplicity alone is not enough: a straight line may miss an obvious curve.

Do not choose a model by its final test score and then report that same score as an untouched evaluation.

Time-series regression

Time itself can supply the features:

$$\hat{T}(t) = \underbrace{\theta_1 + \theta_2 t}_{\text{level and trend}} + \underbrace{\theta_3 \sin(2\pi t) + \theta_4 \cos(2\pi t)}_{\text{yearly cycle}}.$$

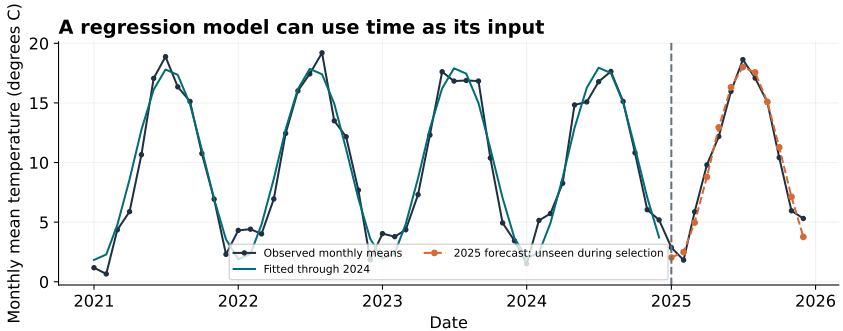
Here t is time in years from a fixed origin. Each observation gives the row

$$A_{i,:} = \big[1,\ t_i,\ \sin(2\pi t_i),\ \cos(2\pi t_i)\big].$$

- Fit the four coefficients using the same least-squares solver.
- To predict a future date, evaluate its feature row and multiply by $\hat{\theta}$.
- Assume the fitted pattern remains useful over that forecast horizon.

This is the seasonal fitting idea from Class 4, now written as linear regression. The trend term is optional; the daily example there used no trend.

Weather



Real Odense data. Each point is a monthly mean. Training: 2006–2023; validation: 2024; test: 2025. Select a feature family on validation, then refit through 2024 before predicting 2025.

Forecast inputs

Past temperatures can also be the features:

$$\hat{T}_{t+1} = \theta_1 + \theta_2 T_t + \theta_3 T_{t-1}.$$

Constant	Today's value	Yesterday's value	Next value (target)
1	5	4	7
1	7	5	6
1	6	7	8

- Each row uses only values available before its target: this is an AR(2) model.
- Later forecast steps reuse predictions if their lagged inputs are not yet observed.

Keep time order in training and validation. This AR model has no moving-average error term; it is not ARMA.

Key ideas

Modelling

Inputs and outcomes

Features and coefficients

Residuals and validation

Computation

Rank and consistency

Least squares and projection

QR, conditioning and `lstsq`

Choose features, build A, fit θ, check predictions.

Readings

Boyd and Vandenberghe: *Introduction to Applied Linear Algebra*

- Boyd and Vandenberghe, §2.3; §13.1: regression notation and data fitting.
- Boyd and Vandenberghe, §8.3; §11.3: linear systems and solving them.
- Boyd and Vandenberghe, §10.4; §12.1–12.3: QR and least squares.
- Boyd and Vandenberghe, §11.5; §13.2–13.3: minimum-norm solutions, validation and feature engineering.

Try-it companion

Class 5 notebook: run the examples and explore LU, QR, least squares, polynomial fits and weather data. The bundle includes the dataset.

Python: NumPy solve, lstsq, QR; SciPy triangular solve.

Weather data: ERA5 via Open-Meteo, CC BY 4.0; source details in the try-it bundle.